

PHỤ LỤC A. HƯỚNG DẪN VÀ ĐÁP ÁN MỘT SỐ BÀI TẬP

Chương 1

Câu 1.

a) Bảng chân trị

p	q	$\neg q$	$p \wedge \neg q$
1	1	0	0
1	0	1	1
0	1	0	0
0	0	1	0

P	q	$p \rightarrow q$	$\neg(p \rightarrow q)$
1	1	1	0
1	0	0	1
0	1	1	0
0	0	1	0

b) Kiểm tra dạng mệnh đề $\neg(p \rightarrow q) \Leftrightarrow (p \wedge \neg q)$ là hằng đúng.

Bước	Qui luật logic
$\neg(p \rightarrow q)$	
$\Leftrightarrow \neg(\neg p \vee q)$	Thay thế tương đương
$\Leftrightarrow \neg(\neg p) \wedge \neg q$	De Morgan
$\Leftrightarrow p \wedge \neg q$	Phủ định của phủ định

Câu 5.

Biểu thức	Qui luật logic
$(q \wedge p) \vee \neg(q \rightarrow p)$	
$\Leftrightarrow (q \wedge p) \vee \neg(\neg q \vee p)$	Thay thế tương đương

$\Leftrightarrow (q \wedge p) \vee (\neg(\neg q) \wedge \neg p)$	De Morgan
$\Leftrightarrow (q \wedge p) \vee (q \wedge \neg p)$	Phủ định của phủ định
$\Leftrightarrow q \wedge (p \vee \neg p)$	Phân phối
$\Leftrightarrow q \wedge 1$	Phần tử bù
$\Leftrightarrow q$	Phần tử trung hòa

Câu 6.

a) Ta có $p(1)=1, p(3)=1, q(1)=1, q(3)=1$ nên với $x=1,3$ ta có $(p(x) \rightarrow q(x))=1$.
 Với $x \neq 1,3, p(x)=0$ nên $(p(x) \rightarrow q(x))=1$. Vậy mệnh đề đã cho là đúng.

b) Ta có $p(0)=0$ nên $(p(0) \rightarrow \neg q(0))=1$. Vậy mệnh đề đã cho là đúng.

c) Xét $\neg(\forall x \in \mathbf{R}, p(x) \rightarrow \neg q(x))$
 $= \exists x \in \mathbf{R}, \neg(p(x) \rightarrow \neg q(x))$
 $= \exists x \in \mathbf{R}, \neg(\neg p(x) \vee \neg q(x))$
 $= \exists x \in \mathbf{R}, p(x) \wedge q(x)$

Ta có $p(1)=1, q(1)=1$ nên $(p(1) \wedge q(1))=1$.

Vậy mệnh đề $(\exists x \in \mathbf{R}, p(x) \wedge q(x))=1$.

Do đó mệnh đề $\forall x \in \mathbf{R}, p(x) \rightarrow \neg q(x)$ là sai.

Câu 7.

$$p \vee s$$

$$\Leftrightarrow p \vee \neg \neg s$$

$$\Leftrightarrow \neg s \rightarrow p$$

$$\neg s$$

$$\therefore p \quad [\text{phương pháp khẳng định}]$$

$$\text{mà } p \rightarrow (q \rightarrow r)$$

$$\therefore q \rightarrow r \quad [\text{phương pháp khẳng định}]$$

$$\begin{aligned}
 & t \rightarrow q \\
 \therefore & t \rightarrow r \quad [\text{tam đoạn luận}] \\
 \Leftrightarrow & \neg t \vee r \\
 \Leftrightarrow & \neg t \vee \neg \neg r \\
 \Leftrightarrow & \neg r \rightarrow \neg t
 \end{aligned}$$

Câu 9.

Bước	Lý do
1. $\neg p \rightarrow q$	Tiền đề
2. $q \rightarrow s$	Tiền đề
3. $\neg p \rightarrow s$	B_1, B_2 và tam đoạn luận
4. $p \rightarrow r$	Tiền đề
5. $\neg r \rightarrow \neg p$	B_4 và tương đương logic
6. $\therefore \neg r \rightarrow s$	B_5, B_3 và tam đoạn luận

Vậy suy luận trên là suy luận đúng.

Câu 10.

Bước	Lý do
1. $\forall x \in A, p(x) \rightarrow q(x)$	Tiền đề
2. $\forall x \in A, q(x) \rightarrow r(x)$	Đặc biệt hoá phổ dụng
3. $p(a) \rightarrow q(a)$	B_1 và đặc biệt hoá phổ dụng
4. $q(a) \rightarrow r(a)$	B_2 và đặc biệt hoá phổ dụng

5. $p(a) \rightarrow r(a)$
6. $\therefore \forall x \in A, p(x) \rightarrow r(x)$
dùng

B2, 4 và tam đoạn luận
B5 và tổng quát hoá phổ

Chương 2

Câu 1.

Phản xạ: c) vì với mọi x trong tập đã cho, $x \geq x$.

Đối xứng : Không có quan hệ nào.

Phản xứng : a), b), c)

Bắc cầu : b), c)

Thứ tự : c)

Câu 2.

- a) $\text{Sup } \{e, b\} = e$
- b) $\text{Inf } \{e, t\} = b$
- c) $\text{Sup } \{d, v\} = x$

Câu 3.

a) Dùng định nghĩa.

b) $[(1,3)] = \{ (1,3), (3,1), (2,2) \}$

$[(2,4)] = \{ (2,4), (4,2), (1,5), (5,1), (3,3) \}$

$[(1,1)] = \{ (1,1) \}$

c) Các lớp tương đương:

$[(1,1)], [(1,2)], [(1,3)], [(2,3)], [(2,4)], [(2,5)], [(3,5)], [(4,5)], [(5,5)].$

Câu 4.

a) $\sup \{b, c\} = e, \quad \inf \{b, c\} = a$

b) $\sup \{a, d, e\} = e, \quad \inf \{a, d, e\} = a$

Câu 5.

a) $\sup \{b, c, d\} = d, \inf \{b, c, d\} = a$

b) $\sup \{b, c\}$ không có, $\inf \{b, c\} = a$

c) $\sup \{d, e\} = f, \inf \{d, e\}$ không có

Chương 3**Câu 1.**

a) Các tế bào lớn : $yzt, xyt, y\bar{z}\bar{t}, \bar{x}\bar{z}\bar{t}, xy\bar{z}$.

Chỉ có một công thức tối thiểu : $f = yzt \vee \bar{x}\bar{z}\bar{t} \vee xy\bar{z}$

b)

Các tế bào lớn : $\bar{x}\bar{z}, \bar{x}y, \bar{y}zt, x\bar{y}\bar{t}, \bar{x}\bar{z}\bar{t}, y\bar{z}\bar{t}$.

Các công thức đa thức tối thiểu:

$$\bar{x}\bar{z} \vee \bar{x}y \vee x\bar{y}\bar{t} \vee \bar{x}\bar{z}\bar{t},$$

$$\bar{x}\bar{z} \vee \bar{x}y \vee x\bar{y}\bar{t} \vee y\bar{z}\bar{t}$$

$$\bar{x}\bar{z} \vee \bar{x}y \vee x\bar{z}\bar{t} \vee \bar{y}zt$$

c)

Các tế bào lớn : $\bar{y}\bar{t}, \bar{z}\bar{t}, \bar{z}y, yt$.

Các công thức đa thức tối thiểu :

$$\overline{y}\overline{t} \vee yt \vee \overline{z}y$$

$$\overline{y}\overline{t} \vee yt \vee \overline{z}\overline{t}$$

d)

Các tế bào lớn : $\overline{x}\overline{z}\overline{t}$, $\overline{x}yz$, $\overline{x}y\overline{t}$, $y\overline{z}\overline{t}$, xyz , $\overline{x}y\overline{t}$.

Các công thức đa thức tối thiểu :

$$xyz \vee \overline{x}\overline{y}\overline{t} \vee \overline{x}yz \vee y\overline{z}\overline{t}$$

$$xyz \vee \overline{x}\overline{y}\overline{t} \vee \overline{x}\overline{z}\overline{t} \vee \overline{x}y\overline{t}$$

$$xyz \vee \overline{x}\overline{y}\overline{t} \vee \overline{x}yz \vee \overline{x}y\overline{t}$$

Chương 4

Câu 2: Không tồn tại đồ thị này, vì số đỉnh bậc lẻ trong đồ thị phải là số chẵn (hệ quả của định lý bắt tay).

Câu 4:

- | | |
|-------|-------|
| a) 28 | b) 12 |
| c) 30 | d) 12 |

Câu 5:

Đáp án: 17 đồ thị con

Câu 6:

Đáp án: Tổng số đồ thị $C_{15}^5 + C_{15}^7$

Hướng dẫn: Đồ thị đầy đủ 6 đỉnh có $6 \cdot (6-1)/2 = 15$ cạnh. Đồ thị có 5 cạnh, 6 đỉnh là một cách chọn không tính thứ tự 5 trong 15 cạnh. Tương tự với đồ thị 7 cạnh, 6 đỉnh.

Câu 8:

Hướng dẫn: Giả sử không tồn tại đường đi nối hai đỉnh bậc lẻ của đồ thị. Như vậy đồ thị có ít nhất hai thành phần liên thông. Hai đỉnh lẻ duy nhất của đồ thị thuộc hai thành phần liên thông khác nhau. Điều này vô lý (xét theo hệ quả của định lý bắt tay, mỗi thành phần liên thông cũng là một đồ thị).

Câu 10:

Hướng dẫn: Trong một đồ thị đơn vô hướng, bậc lớn nhất của một đỉnh là $n-1$. Giả sử không tồn tại hai đỉnh có bậc bằng nhau, như vậy danh sách bậc các đỉnh là: 0, 1, ..., $n-1$. Điều này vô lý, vì nếu tồn tại đỉnh có bậc là $n-1$ thì không thể tồn tại đỉnh có bậc là 0.

Hướng dẫn: Chuyển bài toán thành dạng đồ thị, mỗi đội là một đỉnh, mỗi trận đấu là một cạnh của đồ thị. Áp dụng tính chất cơ bản của đồ thị để chứng minh.

Câu 1:

	1	2	3	4	5	6
1	0	0	0	0	0	0
2	0	0	1	0	1	1
3	0	1	0	1	0	1
4	0	0	1	0	0	0
5	0	1	0	0	0	1
6	0	1	1	0	1	0

	1	2	3	4	5
1	0	1	0	0	2
2	1	0	1	0	1
3	0	1	0	1	0
4	0	0	1	2	1
5	2	1	0	1	0

Định 6: 4, 5

[illegible]

Câu 4:

	1	2	3	4	5	6
1		11	∞	-8	∞	∞
2	∞		∞	0	3	-9
3	-8	0		∞	13	1
4	∞	3	13		∞	6
5	∞	-9	1	6		∞
6	∞	∞	∞	2	0	

Câu 5:

- a) Đẳng cấu
- b) Không đẳng cấu
- c) Đẳng cấu

Chương 6

Câu 1:

- a) Thứ tự duyệt các đỉnh: 0, 1, 7, 2, 6, 9, 3, 4, 5, 8
- b) Thứ tự duyệt các đỉnh: 0, 1, 2, 7, 6, 9, 3, 4, 5, 8

Câu 2:

- a) Thứ tự duyệt các đỉnh: a, b, e, g, c, d, f
- b) Thứ tự duyệt các đỉnh: a, b, g, e, c, d, f

Câu 5:

Hướng dẫn: Xét tất cả các cạnh của đồ thị, thực hiện bỏ 1 cạnh, kiểm tra xem số thành phần liên thông của đồ thị có tăng không. Nếu tăng, cạnh là cầu.

Câu 8:

Hướng dẫn: Điều kiện để một đồ thị là đồ thị vòng: liên thông và tất cả các đỉnh có bậc là 2.

Câu 9:

Hướng dẫn: Chuyển bài toán về dạng đồ thị, mỗi ô là một đỉnh. Khi hai ô có thể đi qua lại với nhau tương ứng là một cạnh. Bài toán trở thành: Đếm số thành phần liên thông của đồ thị, cho biết thành phần liên thông lớn nhất có bao nhiêu đỉnh.

Câu 10:

Hướng dẫn: Chuyển bài toán về dạng đồ thị, mỗi lập trình viên là một đỉnh. Khi hai người có địa chỉ email của nhau tương ứng là một cạnh. Bài toán trở thành: Đếm số thành phần liên thông của đồ thị, cho biết tên 1 đỉnh thuộc mỗi thành phần liên thông.

Chương 7

Câu 1:

- a) Có chu trình Euler.
- b) Không có chu trình Euler.

Câu 2:

- a) Có chu trình Hamilton.
- b) Không có chu trình Hamilton.

Câu 6:

- a) m, n chẵn.
- b) $m=1$ và $n=1$.

Nếu m và n không đồng thời bằng 1: Đồ thị có đường đi Euler khi m chẵn và n lẻ, hoặc m lẻ và n chẵn.

Câu 9:

Hướng dẫn: Chuyển bài toán về dạng đồ thị. Mỗi ô trên bàn cờ là 1 đỉnh. Các ô mà quân mã có thể di chuyển qua lại tương ứng là một cạnh. Chuyển toàn bộ thông tin trên bàn cờ thành một đồ thị. Việc xác định lộ trình của quân mã để đi qua tất cả các ô tương ứng với việc xác định chu trình Hamilton của đồ thị.

Chương 8

Câu 1:

Số lá: 75 đỉnh

Câu 2:

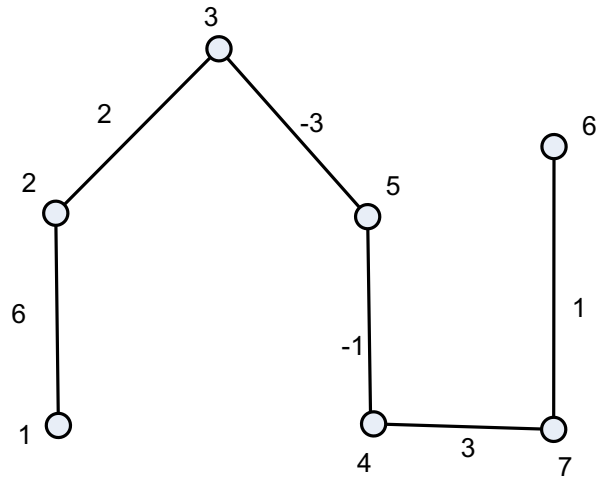
Số cạnh cần xóa: $m-n+1$

Câu 3:

Khi cạnh đó là cầu

Câu 5:

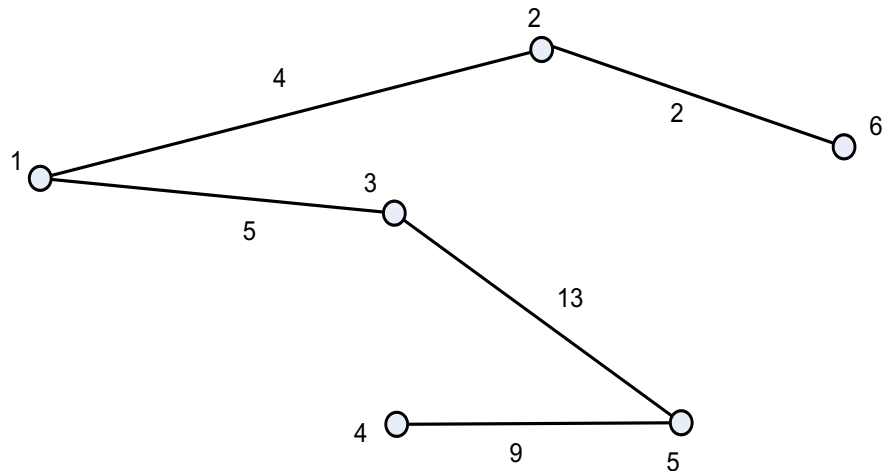
Cây khung nhỏ nhất là:



Tổng trọng số: 8

Câu 6:

Cây khung nhỏ nhất là:



Tổng trọng số: 33.

Câu 7:

Hướng dẫn: Kiểm tra tính liên thông và có hay không có chu trình của đồ thị.

Chương 9

Câu 1:

Tham khảo chứng minh định lý đồ thị 2-màu

Câu 2:

- a) 3 màu
- b) 3 màu

Câu 3:

Nếu n chẵn: 3 màu

Nếu n lẻ: 4 màu

Chương 10

Câu 1:

	d[b], Truoc[b]	d[c], Truoc[c]	d[d], Truoc[d]	d[e], Truoc[e]	d[f], Truoc[f]	d[g], Truoc[g]
1	3, a	12, a	∞ , a	∞ , a	∞ , a	∞ , a
2	3, a	12, a	-1, b	∞ , a	∞ , a,	16, c
3	3, a	12, a	-1, b	18, d	14, d	5, d
4	3, a	12, a	-1, b	16, f	12, g	5, d
5	3, a	12, a	-1, b	14, f	12, g	5, d
6	3, a	12, a	-1, b	14, f	12, g	5, d

Câu 2:

	d[b], Truoc [b]	d[c], Truoc [c]	d[d], Truoc [d]	d[e], Truoc [e]	d[f], Truoc [f]	d[g], Truoc [g]	d[h], Truoc [h]
1	3, a	∞ , a	15, a	∞ , a	∞ , a	∞ , a	∞ , a
2	-	9, b	8, b	12, b	∞ , a	17, b	∞ , a
3	-	9, b	-	11, d	∞ , a	17, b	∞ , a
4	-	-	-	11, d	∞ , a	13, c	∞ , a
5	-	-	-	-	∞ , a	13, c	13, e
6	-	-	-	-	20, g	-	13, e
7	-	-	-	-	20, g	-	-
	3, a	9, b	8, b	11, d	20, g	3, c	13, e

Câu 3:

Đáp án:

5	-2	-1
8	6	7
6	4	5

2	0	0
1	0	1
2	0	2

Ma trận D

Ma trận P

Câu 7:

Hướng dẫn: Chuyển bài toán về dạng đồ thị, mỗi tỉnh, thành phố là một đỉnh, mỗi con đường là một cạnh, giá trị chi phí đi lại là trọng số của cạnh. Bài toán trở thành tìm đường đi ngắn nhất từ một đỉnh đến các đỉnh còn lại của đồ thị.

Chương 11

Câu 1:

Luồng cực đại: $f=27$.

Tìm được lát cắt hẹp nhất (X, X^*) với: $X=\{s, a, b, c, d\}$, $X^*=\{t\}$

PHỤ LỤC B. CHƯƠNG TRÌNH MẪU

1. Thuật toán Prim tìm cây khung nhỏ nhất

```
//Cài đặt thuật toán PRIM, sử dụng ma trận trọng số
#include<stdio.h>
#include<conio.h>
#define maxver 50
//Khai báo cấu trúc cạnh
typedef struct
{
    int Ver1;
    int Ver2;
    int W;
}ARC;

//Khai báo cấu trúc đồ thị
typedef struct
{
    int nVer;//Số đỉnh
    unsigned int a[maxver][maxver];
    unsigned char TrongV[maxver];
    ARC T[maxver];//Danh sách cạnh
    int nT;
    int dem;
}GRAPH;

//Danh sách các hàm con
void nhap(GRAPH &g);
void xuat(GRAPH &g);
void initData(GRAPH &g);
int searchArc(GRAPH &g,ARC &e,int &x,int &y);
int searchMinArc(GRAPH&g,ARC &e,int &x,int &y);
int PrimAlg(GRAPH &g);

//Hàm main
void main()
```

```
{  
    GRAPH gr;  
    nhap(gr);  
    xuất(gr);  
}
```

//Hàm đọc đồ thị từ tập tin

```
void nhap(GRAPH &g)  
{  
    FILE *f;  
    f=fopen("Duong dan den tap tin input", "rt");  
    fscanf(f, "%d", &g.nVer);  
    for(int i=0; i<g.nVer; i++)  
    {  
        for(int j=0; j<g.nVer; j++)  
            fscanf(f, "%d", &g.a[i][j]);  
        fscanf(f, "\n");  
    }  
    fclose(f);  
}
```

//Hàm khởi tạo

```
void initData(GRAPH &g)  
{  
    g.nT=0; g.dem=0;  
    g.TrongV[0]=1;  
    for(int i=1; i<g.nVer; i++)  
        g.TrongV[i]=0;  
}
```

//Hàm tìm 1 cạnh thỏa mãn điều kiện 1 đỉnh thuộc tập V, 1 đỉnh không thuộc tập V //trong danh sách cạnh

```
int searchArc(GRAPH &g, ARC &e, int &x, int &y)  
{  
    for(int i=0; i<g.nVer; i++)  
        if(g.TrongV[i]==1)
```

```

        for(int j=0;j<g.nVer;j++)
            if(g.TrongV[j]==0)
            {
                if(g.a[i][j]!=0)
                {
                    x=i;y=j;
                    e.Ver1=i;
                    e.Ver2=j;
                    e.W=g.a[i][j];
                    return 1;
                }
                if(g.a[j][i]!=0)
                {
                    x=j;y=i;
                    e.Ver1=j;
                    e.Ver2=i;
                    e.W=g.a[j][i];
                    return 1;
                }
            }

    return 0;
}

```

//Tìm cạnh có trọng số nhỏ nhất trong các cạnh thỏa mãn điều kiện 1 đỉnh thuộc tập //V, 1 đỉnh không thuộc tập V.

```

int searchMinArc(GRAPH &g,ARC &e,int &x,int &y)
{
    int min,i,j;
    if(searchArc(g,e,x,y)==0)
        return 0;//ko co canh
    min=e.W;
    for(i=0;i<g.nVer;i++)
        if(g.TrongV[i]==1)
            for(j=0;j<g.nVer;j++)
                if(g.TrongV[j]==0)

        if((g.a[i][j]!=0)&&(g.a[i][j]<min))

```

```

        {
            min=g.a[i][j];
            x=i;
            y=j;
            e.W=g.a[i][j];
            e.Ver1=i;
            e.Ver2=j;
        }

    return 1;
}

```

//Giải thuật chính

```

int PrimAlg(GRAPH& g)
{
    initData(g);
    while(g.nT<g.nVer-1)
    {
        ARC e;
        int x,y;
        if(searchMinArc(g,e,x,y)==0)
            return 0;
        g.dem=g.dem+e.W;
        g.T[g.nT]=e;
        g.TrongV[y] = 1;
        (g.nT)++;
    }
    return 1;
}

```

//Hàm xuất kết quả ra tập tin

```

void xuat(GRAPH &g)
{
    FILE *f;
    int i;
    f=fopen("Duong dan den tap tin output", "wt");
    if(PrimAlg(g)==0)
    {

```



```

        fprintf(f, "\nDo thi khong co cay khung!\n");
    }
    else
    {
        fprintf(f, "Tong trong so:%d", g.dem);

        fprintf(f, "\n%d", g.nT);
        fprintf(f, "\n");
        for(i=0; i<g.nT; i++)
        {
            fprintf(f, "%2d---> %2d", g.T[i].Ver1+1,
g.T[i].Ver2+1);
            fprintf(f, "\n");
        }
    }
    fclose(f);
}

```

2. Thuật toán Kruskal tìm cây khung nhỏ nhất

```

//Cài đặt thuật toán KRUSKAL
//Duyệt theo danh sách cạnh (sắp xếp cạnh theo thứ tự trọng số
tăng dần)
#include<stdio.h>
#include<conio.h>
#include<dos.h>
const    MAXVER=50;
const    MAXEDGE=100;

//Khai báo cấu trúc cạnh
typedef struct
{
    int    Ver1;
    int    Ver2;
    int    w;
}ARC;

//Khai báo cấu trúc đồ thị
typedef struct

```

```

{
    int    nVer;
    int    L[MAXVER][MAXVER];
    ARC    T[MAXVER-1];    //cay khung nho nhat
    int    nT;
    ARC    DS[MAXEDGE];    //danh sach canh
    int    nDS;
}GRAPH;

```

//Các hàm con

```

void    Input(GRAPH&);
void    Output(GRAPH&);
void    ReadEdgeList(GRAPH&);
void    Sort(GRAPH&);
int     Kruskal(GRAPH&);

```

//Hàm main

```

void    main()
{
    GRAPH    gr;
    Input(gr);
    Output(gr);
}

```

//Hàm đọc đồ thị từ tập tin

```

void    Input(GRAPH&    g)
{
    FILE *f;
    f=fopen("Duong dan den tap tin input", "rt");
    fscanf(f, "%d", &g.nVer);
    for(int i=0; i<g.nVer; i++)
    {
        for(int j=0; j<g.nVer; j++)
            fscanf(f, "%5d", &g.L[i][j]);
        fscanf(f, "\n");
    }
    fclose(f);
}

```

```

    }

    //Hàm xuất kết quả ra tập tin
    void Output(GRAPH& g)
    {
        FILE* f;
        f=fopen("Duong dan den tap tin output", "wt");
        fprintf(f, "%d", g.nVer);
        fprintf(f, "\n");
        for(int i=0; i<g.nVer; i++)
        {
            for(int j=0; j<g.nVer; j++)
                fprintf(f, "%5d", g.L[i][j]);
            fprintf(f, "\n");
        }
        Sort(g);
        fprintf(f, "Danh sach canh sau khi da sap tang:\n");
        for(i=0; i<g.nDS; i++)
        {
            fprintf(f, "%d-->%d:%d", g.DS[i].Ver1+1, g.DS[i].Ver2+1, g.DS[i].w);
            fprintf(f, "\n");
        }
        if(Kruskal(g)==0)
            fprintf(f, "\nDo thi khong lien thong\n");
        else
        {
            fprintf(f, "Danh sach canh trong do thi trong so nho\n");
            for(i=0; i<g.nT; i++)
            {
                fprintf(f, "%d-->%d:%d", g.T[i].Ver1+1, g.T[i].Ver2+1, g.T[i].w);
                fprintf(f, "\n");
            }
        }
        fclose(f);
    }
}

```

//Hàm khởi tạo

```
void InitData(GRAPH& g)
```

```
{
```

```
    g.nT=0;
```

```
}
```

//Hàm xây dựng danh sách cạnh từ mà trận trọng số

```
void ReadEdgeList(GRAPH& g)
```

```
{
```

```
    g.nDS=0;
```

```
    for(int i=0; i<g.nVer; i++)
```

```
        for(int j=i; j<g.nVer; j++)
```

```
        {
```

```
            if(g.L[i][j]!=0)//(g.L[j][i]))
```

```
            {
```

```
                g.DS[g.nDS].Ver1=i;
```

```
                g.DS[g.nDS].Ver2=j;
```

```
                g.DS[g.nDS].w=g.L[j][i];
```

```
                g.nDS++;
```

```
            }
```

```
        }
```

```
}
```

//Hàm sắp xếp danh sách cạnh theo thứ tự trọng số tăng dần

```
void Sort(GRAPH& g)
```

```
{
```

```
    ReadEdgeList(g);
```

```
    ARC tam;
```

```
    for(int i=0; i<g.nDS-1; i++)
```

```
        for(int j=i+1; j<g.nDS; j++)
```

```
            if(g.DS[i].w>g.DS[j].w)
```

```
            {
```

```
                tam=g.DS[i];
```

```
                g.DS[i]=g.DS[j];
```

```
                g.DS[j]=tam;
```

```
            }
```

```

}

//Giải thuật chính
int    Kruskal(GRAPH& g)
{
    int i,j;
    int    Sohieu[MAXVER];
    InitData(g);
    //Gan nhan
    for(i=0; i<g.nVer; i++)
        Sohieu[i]=i;
    j=0;
    do
    {
        if(Sohieu[g.DS[j].Ver1]!=Sohieu[g.DS[j].Ver2])
        {
            //Dua canh j vao cay T
            g.T[g.nT].Ver1=g.DS[j].Ver1;
            g.T[g.nT].Ver2=g.DS[j].Ver2;
            g.T[g.nT].w=g.DS[j].w;

            //Cap nhat lai so hieu
            int x=Sohieu[g.DS[j].Ver2];
            int y=Sohieu[g.DS[j].Ver1];
            for(int l=0; l<g.nVer; l++)
            {
                if(Sohieu[l]==y)
                    Sohieu[l]=x;
            }
            g.nT++;
        }
        j++;
    } while ((g.nT<g.nVer)&&(j<g.nDS));
    if(g.nT<g.nVer-1)    return  0;
    return  1;
}

```

3. Thuật toán Dijkstra tìm đường đi ngắn nhất từ một đỉnh đến các đỉnh còn lại của đồ thị

```
//Cài đặt thuật toán DIJKSTRA
#include<stdio.h>
#include"conio.h"
#define MAX 100
#define VOCUC -1

//Khai báo cấu trúc đồ thị
typedef struct
{
    int n;
    int a[MAX][MAX];
}GRAPH;
int T[MAX];
int arrnCost[MAX];
int arrnPrevious[MAX];
int nStartNode;
int nStopNode;

//Các hàm con
void Input(GRAPH &g);
void InitData(GRAPH &g);
int Dijkstra(GRAPH &g);
void PrintResult(GRAPH &g);

//Hàm main
void main()
{
    GRAPH g;
    Input(g);
    printf("Nhap vao dinh bat dau ");
    scanf("%d",&nStartNode);
    nStartNode-=1;
    printf("Nhap vao dinh ket thuc ");
    scanf("%d",&nStopNode);
    nStopNode-=1;
    PrintResult(g);
}
```

//Hàm đọc đồ thị từ tập tin

```
void Input(GRAPH &g)
{
    FILE *f;
    f=fopen("Duong dan den tap tin input", "rt");
    fscanf(f, "%d", &g.n);
    for(int i=0; i<g.n; i++)
        for(int j=0; j<g.n; j++)
            fscanf(f, "%d", &g.a[i][j]);
    fclose(f);
}
```

//Hàm khởi tạo dữ liệu

```
void InitData(GRAPH &g)
{
    for(int i=0; i<g.n; i++)
    {
        T[i]=1;
        if(i!=nStartNode)
            arrnCost[i]= VOCUC;
    }
    arrnCost[nStartNode]=0;
}
```

//Giải thuật chính

```
int Dijkstra(GRAPH &g)
{
    InitData(g);
    int v=-1;
    int c=0;
    int min;
    while(v!=nStopNode)
    {
        min=32767;
        for(int i=0; i<g.n; i++)
            if(T[i]==1 && min > arrnCost[i] && arrnCost[i]!=VOCUC)
            {
                min=arrnCost[i];
            }
    }
}
```

```

        v=i;
        c=1;
    }
    if(c==0) {return 0; break;}
    T[v] = 0;
    for(int k=0;k<g.n;k++)

        if(T[k]==1 && g.a[v][k]!=0 && (arrnCost[k]==VOCUC ||
            (arrnCost[k] > arrnCost[v] + g.a[v][k])))
        {
            arrnCost[k] = arrnCost[v] + g.a[v][k];
            arrnPrevious[k]=v;
        }
    }
    return 1;
}

//Hàm xuất kết quả ra màn hình
void PrintResult(GRAPH &g)
{
    int k;
    if(Dijkstra(g)==0) printf("khong co duong di");
    else
    {
        printf("\n Chi phi toi uu la: %d",arrnCost[nStopNode]);
        printf("\n Duong di tim duoc: \n");
        k=nStopNode;
        printf("%d ",k+1);
        do
        {
            k=arrnPrevious[k];
            printf(" <---- %d ", k+1);
        } while(k!=nStartNode);
    }
}

```

4. Thuật toán Floyd tìm đường đi ngắn nhất giữa tất cả các cặp đỉnh của đồ thị

```
#include<stdio.h>
```



```

#define MAX 100
struct GRAPH
{
    int n;
    int a[MAX][MAX];
};
int d[MAX][MAX];
int p[MAX][MAX];
//Hàm nhập đồ thị
void Input(GRAPH &g)
{
    FILE* f;
    f = fopen("Duong dan den tap tin input", "rt");
    if (f == NULL)
    {
        printf("Khong mo duoc file\n");
        return;
    }
    fscanf(f, "%d", &g.n);
    for (int i=0; i<g.n; i++)
        for (int j=0; j<g.n; j++)
            fscanf(f, "%d", &g.a[i][j]);
    fclose(f);
}
//Hàm xuất đồ thị ra màn hình
void Output(GRAPH &g)
{
    printf("%d\n", g.n);
    int i, j;
    for (i=0; i<g.n; i++)
    {
        for (j=0; j<g.n; j++)
            printf("%5d", g.a[i][j]);
        printf("\n");
    }
}
//Hàm xuất kết quả hai ma trận d[][] và p[][]
void XuatKQ(GRAPH g)

```

```

{
    printf("\n");
    for(int i=0;i<g.n;i++)
    {
        for(int j=0;j<g.n;j++)
            printf("%10d",d[i][j]);
        printf("\n");
    }
    printf("\n");
    for(int i=0;i<g.n;i++)
    {
        for(int j=0;j<g.n;j++)
            printf("%10d",p[i][j]);
        printf("\n");
    }
}
//Giải thuật chính Floyd
void Floyd(GRAPH& g)
{
    for (int i = 0; i <g.n; i++) /* Khởi tạo */
        for (int j = 0; j <g.n; j++)
        {
            d[i][j] = g.a[i][j];
            p[i][j] = i;
        }
    for (int k = 0; k < g.n; k++) /* 3 vòng lặp */
        for (int i = 0; i < g.n; i++)
            for (int j = 0; j < g.n; j++)
                if (d[i][j] > d[i][k] + d[k][j])
                {
                    d[i][j] = d[i][k] + d[k][j];
                    p[i][j] = p[k][j];
                }

    XuatKQ(g);
}
//Hàm main
void main()
{

```

```
GRAPH g;
```

```
Input(g);
```

```
Output(g);
```

```
Floyd(g);
```

```
}
```

